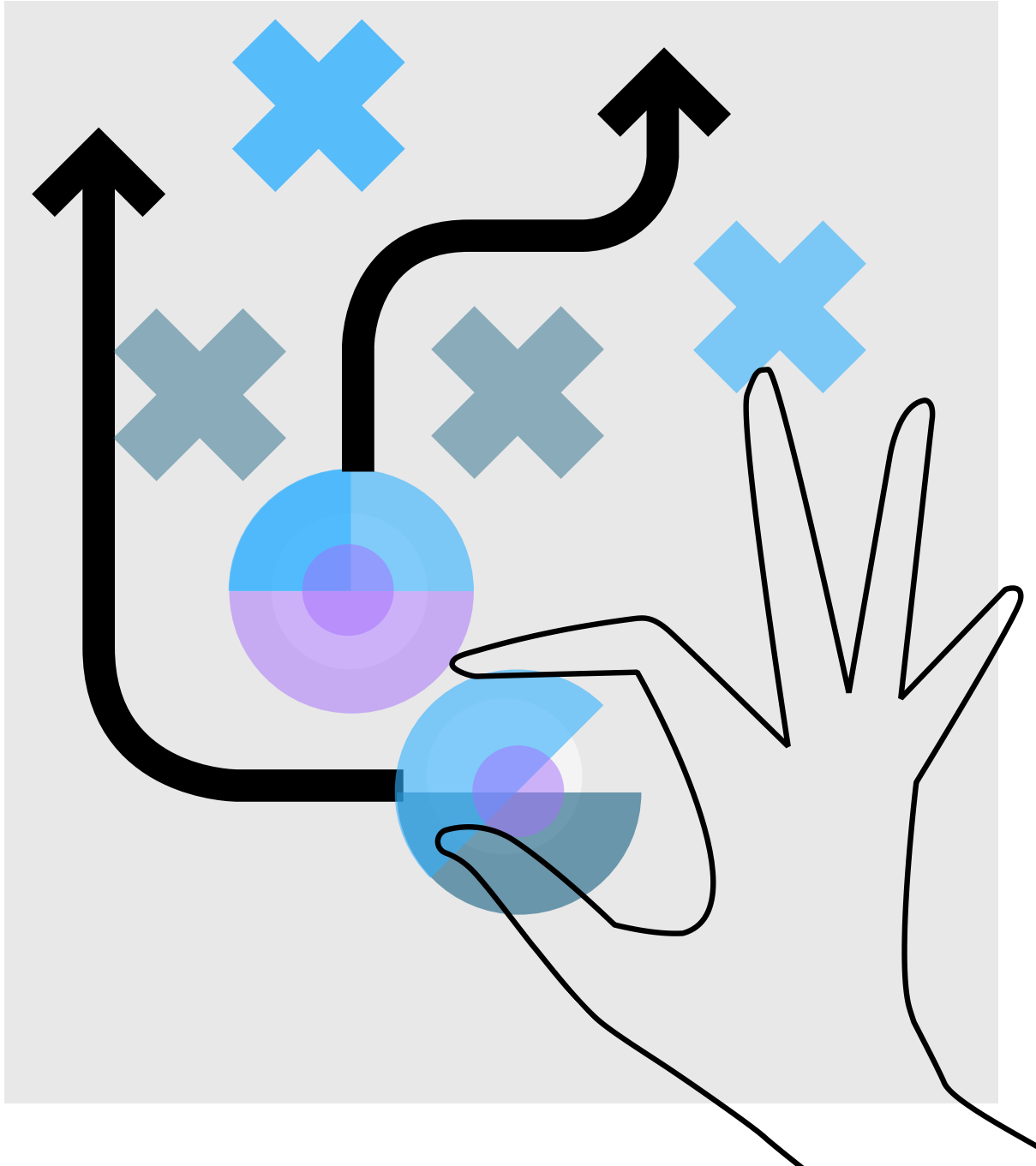
遠東金士頓

SuccessFactors Project

SAP BTP 的系統連線架構與技術機制



IBM Consulting, Talent Transformation

2026.07.22

講師

課程表		課程內容備註
第一堂	預計授課日期：yyyymmdd	
	課程：BTP架構與核心概念 . BTP Global Account / Subaccount . Region / Environment . Entitlement . Service Plan . Multi-cloud architecture (AWS/Azure/GCP)	1. BTP是甚麼(Platform定位) 2. BTP四大能力 3. BTP架構 4. Account Model 概念介紹 5. BTP環境介紹 6. Extension Strategy
	熟悉 BTP Cockpit 畫面布局	
	學習材料、網站閱讀	
第二堂	預計授課日期：yyyymmdd	
	課程：BTP Account Model . Global Account . Directory . Subaccount . Quota/Entitlement	1. BTP帳號層級結構 2. 資源如何分配 3. 專案與系統如何隔離 4. 權限與管理方式
	建立Subaccount，分配Entitlement	
	學習材料、網站閱讀	
第三堂	預計授課日期：yyyymmdd	
	課程：BTP Environment . Cloud Foundry . Kyma . ABAP Environment . Neo(legacy)	1. BTP的Runtime Environment概念與架構 2. 各種Environment的用途與適用場景說明
	建立Cloud Foundry environment Org/Space	
	學習材料、網站閱讀	
第四堂	預計授課日期：yyyymmdd	
	課程：BTP Security . Identity Provider . Trust configuration . Role Collection . XSUAA	1. 介紹SAP BTP的安全架構與身份管理機制 2. 了解如何在雲端環境中管理使用者與權限
	建立 Role collection / SSO Login	
	學習材料、網站閱讀	
第五堂	預計授課日期：yyyymmdd	
	課程：BTP Connectivity . Destination service . Connectivity service . Cloud Connector	介紹 SAP BTP 的系統連線架構與技術機制，說明如何安全地讓 BTP 應用程式存取： . On-Premise SA統 . Cloud application . 外部 API
	建立Destination / 連接 On-Premise System	
	學習材料、網站閱讀	
第六堂	預計授課日期：yyyymmdd	
	課程：BTP Service Lifecycle . Service instance . Service binding . Service key	介紹 BTP 服務的生命週期管理（Service Lifecycle），說明如何： . 啟用服務 . 建立 Service Instance . 將服務綁定到應用程式 . 管理服務資源
	建立Service instance	
	學習材料、網站閱讀	

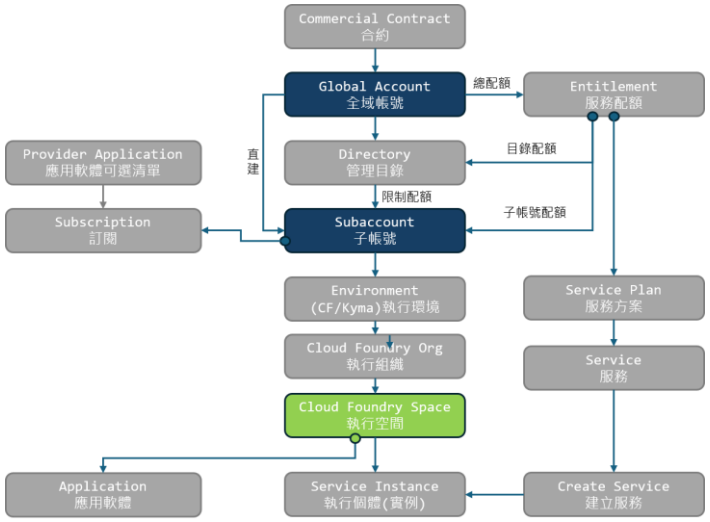
Topic

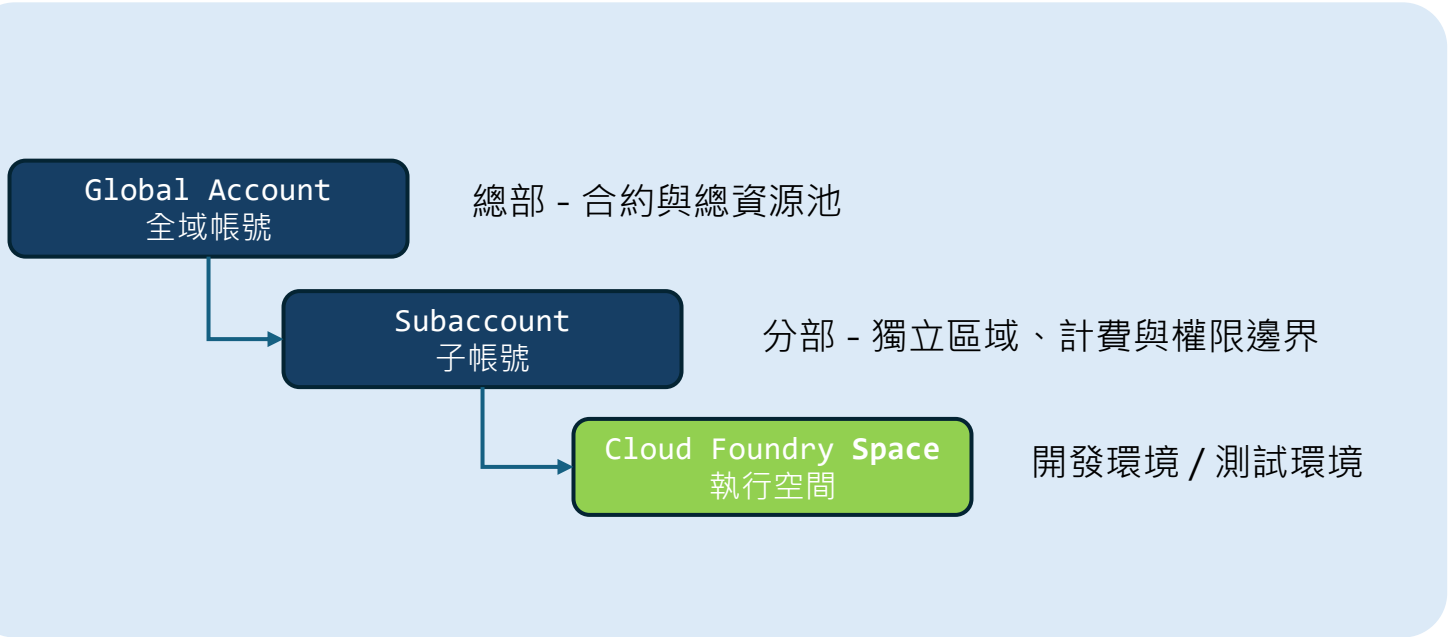
1. SAP BTP 平台架構與環境基礎
2. 服務實例 (Service Instances) 與生命週期管理
3. 解密應用連線：Service Bindings 與 Service Keys
4. 跨系統整合架構：Connectivity 與 Destination 服務)
5. 雲地整合橋樑：SAP Cloud Connector 技術解析
6. 身分驗證與安全防護機制 (IAM)

1. SAP BTP 平台架構與環境基礎

建立 BTP 空間架構的清晰藍圖，理解資源是如何被隔離與管理的。

- **BTP 階層結構：Global Account、Subaccount、Space 的關係與資源分配。**
- 多雲運行環境對比：Cloud Foundry、Kyma (Kubernetes) 與 ABAP 環境的架構差異。
- 服務市場 (Marketplace)：服務方案 (Service Plans) 與授權配額 (Entitlements) 的運作邏輯。

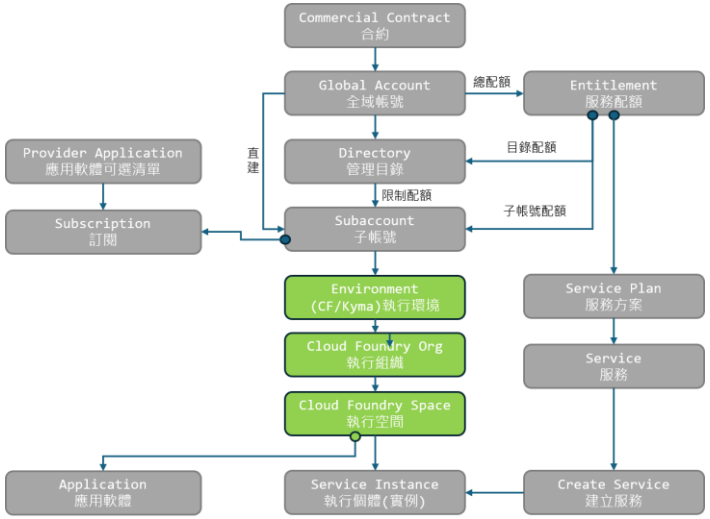




1. SAP BTP 平台架構與環境基礎

建立 BTP 空間架構的清晰藍圖，理解資源是如何被隔離與管理的。

- BTP 階層結構：Global Account、Subaccount、Space 的關係與資源分配。
- **多雲運行環境對比：Cloud Foundry、Kyma (Kubernetes) 與 ABAP 環境的架構差異。**
- 服務市場 (Marketplace)：服務方案 (Service Plans) 與授權配額 (Entitlements) 的運作邏輯。

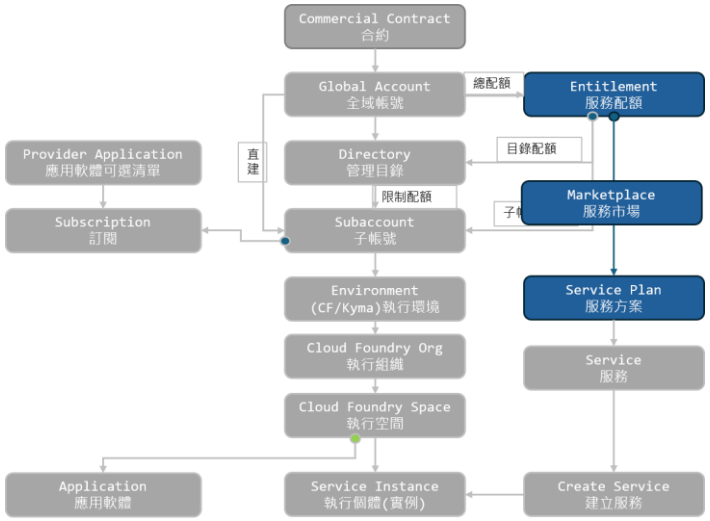


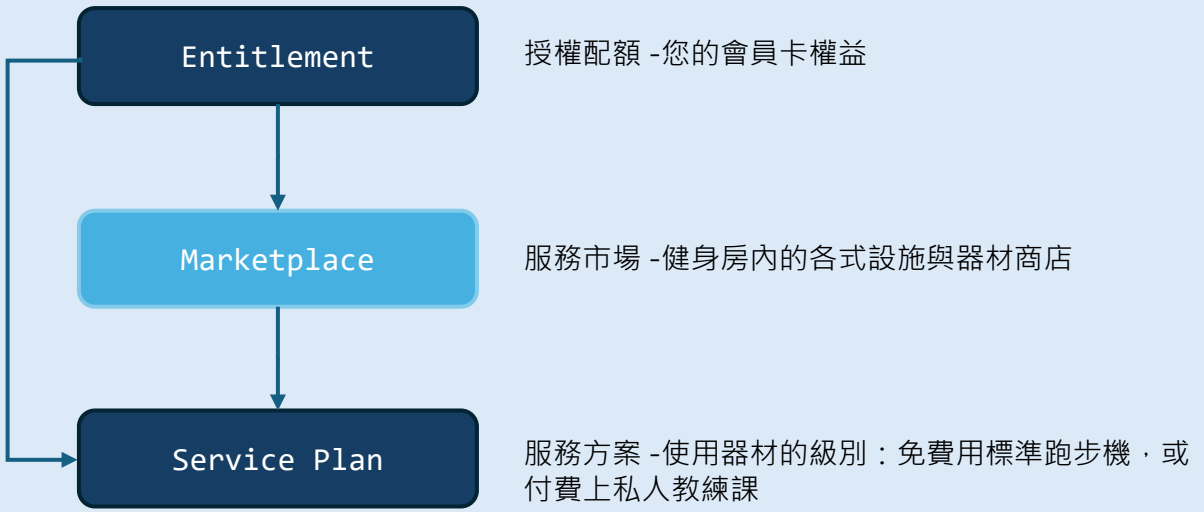
運行環境	Cloud Foundry (CF)	Kyma (Kubernetes)	ABAP
技術比喻	精裝商務公寓（家具齊全）	自建透天厝（高度客製化）	百年古董飯店(專屬高貴)
核心技術	Buildpacks (自動打包程式碼)	Docker 容器與 Kubernetes (K8s)	SAP 專屬的 ABAP 核心
開發語言	Java, Node.js, Python, Go 等	任何能封裝進容器的語言	嚴格限定為 ABAP 語言
運維負載	**極低**。開發者只需專注寫程式碼，平台自動處理基礎架構。	**中到高**。需自行管理 K8s 的 Pod、Ingress 與微服務網路結構。	**低**。由 SAP 自動維護與升級底層的核心系統。

1. SAP BTP 平台架構與環境基礎

建立 BTP 空間架構的清晰藍圖，理解資源是如何被隔離與管理的。

- BTP 階層結構：Global Account、Subaccount、Space 的關係與資源分配。
- 多雲運行環境對比：Cloud Foundry、Kyma (Kubernetes) 與 ABAP 環境的架構差異。
- **服務市場 (Marketplace)：服務方案 (Service Plans) 與授權配額 (Entitlements) 的運作邏輯。**



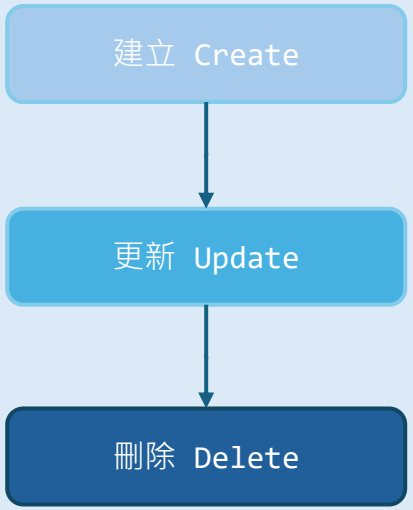


2.服務實例 (Service Instances) 與生命週期管理

BTP 資源的生成與配置，如何動態管理平台上的各類服務。

- **服務實例的本質：如何透過 BTP Cockpit 與 CLI 建立、更新與刪除服務實例。**
- HDI Container 機制：以 SAP HANA Cloud 為例，理解獨立 Schema 的隔離與動態配置。
- 共享與獨立資源：區分多租戶（Multi-tenant）服務與專屬實例的資源配置。

Service Instances Life Cycle



平台在雲端配置資源

Service -> Marketplace -> Create 服務方案 + 實例名稱

cf create-service destination lite my-erp-link

動態調整規格、方案或綁定參數

cf update-service my-erp-link -c '{"existing_destinations_policy": "update"}'

釋放資源，停止計費

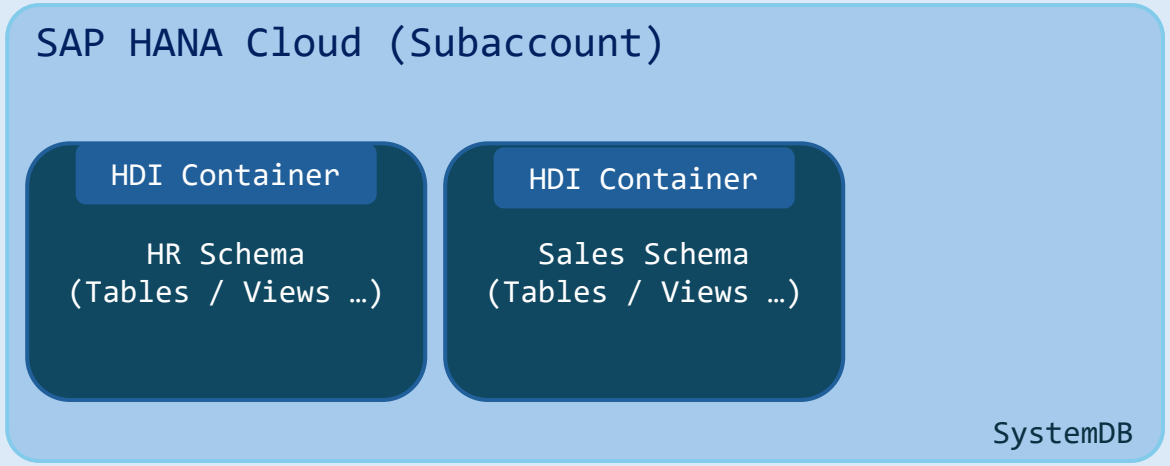
cf delete-service my-erp-link

2.服務實例 (Service Instances) 與生命週期管理

BTP 資源的生成與配置，如何動態管理平台上的各類服務。

- 服務實例的本質：如何透過 BTP Cockpit 與 CLI 建立、更新與刪除服務實例。
- **HDI Container 機制：以 SAP HANA Cloud 為例，理解獨立 Schema 的隔離與動態配置。**
- 共享與獨立資源：區分多租戶（Multi-tenant）服務與專屬實例的資源配置。

HDI（HANA Deployment Infrastructure）Container 是 SAP HANA Cloud 中最核心的隔離技術。



運作本質：

- 每個 HDI Container 在底層都是一個獨立的 Database Schema（資料庫綱要）。
- 它包含了自己專屬的資料表（Tables）、視圖（Views）和預存程序。
- 可以透過 Service Binding 提供讀寫權限給其他 subaccount 或 Apps

2.服務實例 (Service Instances) 與生命週期管理

BTP 資源的生成與配置，如何動態管理平台上的各類服務。

- 服務實例的本質：如何透過 BTP Cockpit 與 CLI 建立、更新與刪除服務實例。
- HDI Container 機制：以 SAP HANA Cloud 為例，理解獨立 Schema 的隔離與動態配置。
- **共享與獨立資源：區分多租戶（Multi-tenant）服務與專屬(Dedicated)實例的資源配置。**

Resource Allocation : Multi-tenant、Dedicated

多租戶（Multi-tenant）實例：**例子**：訂閱 **SAP Integration Suite (CPI)** 的 Standard 方案。

- 多個不同的企業客戶（或你同公司內不同的子帳號）底層其實都跑在 SAP 維護的同一個大型伺服器群集上。當中午大家都塞爆 API 傳送報表時，你的傳輸速度可能會稍微變慢（就像捷運尖峰時段上下車比較擠）。

專屬資源（Dedicated）實例：**例子**：建立一個 **SAP HANA Cloud** 的 Production 資料庫。

- SAP 會真正在 AWS 或 Azure 幫你劃分出獨立的 4 核 CPU、32 GB 記憶體。不管全世界其他企業的資料庫怎麼塞爆，你的 HANA Cloud 依運算速度依然全速前進。相對地，就算你整個月沒半個人進去查資料，每小時的租金依然一分不少。
- Integration Suite 提供Dedicated特別方案。

3. 解密應用連線：Service Bindings 與 Service Keys

安全地獲取憑證並與服務進行溝通。

- **Service Binding 運作原理：解密 VCAP_SERVICES 環境變數的注入機制與 JSON 結構。**
- MTA (Multi-Target Application) 自動化綁定：如何在 mta.yaml 中定義 requires 與 resources 的依賴關係。
- Service Key 應用場景：本機開發（VS Code/BAS）、外部工具（Postman/DBeaver）的憑證核發與安全規範。
- 架構對比：Binding 與 Service Key 在安全性、生命週期與適用對象的全面對比。

VCAP_SERVICES → Variable CAP Services

- **VCAP_SERVICES 是 Cloud Foundry 環境中一個非常核心的環境變數（Environment Variable）。**
- **它的主要作用，就是把資料庫（如先前提到 HDI Container）、金鑰、訊息佇列或第三方服務的連線資訊（URL/Host）、帳號、密碼及憑證，安全地注入到您的應用程式中。**

json

```
{
  "destination": [  // 服務類型名稱
    {
      "name": "my-destination-instance", // 服務實例名稱
      "label": "destination",
      "tags": ["destination", "connectivity"],
      "credentials": {  // 🔒 核心憑證區：這才是程式要拿的鑰匙
        "clientid": "sb-clone-abc123xyz...",
        "clientsecret": "p@ssword123!_secret...",
        "url": "https://ondemand.com",
        "uri": "https://ondemand.com"
      }
    }
  ]
}
```

3. 解密應用連線：Service Bindings 與 Service Keys

安全地獲取憑證並與服務進行溝通。

- Service Binding 運作原理：解密 VCAP_SERVICES 環境變數的注入機制與 JSON 結構。
- **MTA (Multi-Target Application) 自動化綁定：如何在 mta.yaml 中定義 requires 與 resources 的依賴關係。**
- Service Key 應用場景：本機開發（VS Code/BAS）、外部工具（Postman/DBeaver）的憑證核發與安全規範。
- 架構對比：Binding 與 Service Key 在安全性、生命週期與適用對象的全面對比。

MTA Multi-Target Application

當專案包含多個模組（網頁前端、後端服務、資料庫）時，手動在網頁點擊綁定太低效。

- **SAP 使用 mta.yaml（Multi-Target Application）描述檔，透過 resources（提供者）與 requires（需求者）的連連看機制，在部署時自動完成建立與綁定。**

yaml

```yaml
_schema-version: "3.1"
ID: my-sap-app
version: 1.0.0

modules:
  - name: my-backend-srv    # 後端 App 模組
    type: nodejs
    path: srv
    requires:
      - name: db-res        # 👉 【連連看 1】宣告：我的後端程式需要連資料庫

resources:
  - name: db-res            # 👉 【連連看 2】宣告：請 BTP 幫我建立一個資料庫實例
    type: com.sap.xs.hdi-container
    parameters:
      service: hana
      service-plan: hdi-shared
```

3. 解密應用連線：Service Bindings 與 Service Keys

安全地獲取憑證並與服務進行溝通。

- Service Binding 運作原理：解密 VCAP_SERVICES 環境變數的注入機制與 JSON 結構。
- MTA (Multi-Target Application) 自動化綁定：如何在 mta.yaml 中定義 requires 與 resources 的依賴關係。
- **Service Key 應用場景：本機開發（VS Code/BAS）、外部工具（Postman/DBeaver）的憑證核發與安全規範。**
- 架構對比：Binding 與 Service Key 在安全性、生命週期與適用對象的全面對比。

Service Key（服務金鑰） 是為了「非 BTP 運行環境」所開的後門。當程式沒有部署在 BTP 容器內（因此無法享受自動注入環境變數），卻又需要連線時，就必須手動核發 Service Key。

- **場景一：本機開發（VS Code / Business Application Studio）應用：工程師在筆電上開發 Node.js 專案，還沒部署到雲端。為了在本機測試連線，會在 BTP Cockpit 對該服務建立一個 Service Key，複製其 JSON 內容，貼到本機專案的 .env 或 default-env.json 檔案中，模擬雲端的環境變數。**
- **場景二：外部工具測試（Postman / DBeaver）應用：測試人員想用 Postman 測試 BTP 上的 API。他們會為服務建立 Service Key，從中取得 clientid、clientsecret 與 url，在 Postman 中設定 OAuth 2.0 驗證，成功換取 Token 以進行 API 呼叫。如果是 HANA 資料庫，則是用 DBeaver 讀取 Service Key 裡的資料庫主機網址（Host）、通訊埠（Port）與帳密來直接連線看資料表。**

安全規範：Service Key 是靜態且永久有效的憑證。強烈禁止將 Service Key 提交（Commit）到 GitHub 等版控系統。一旦人員離職或專案交接，應立即在 BTP 側點擊 Delete 註銷該金鑰，使其立刻失效。

3. 解密應用連線：Service Bindings 與 Service Keys

安全地獲取憑證並與服務進行溝通。

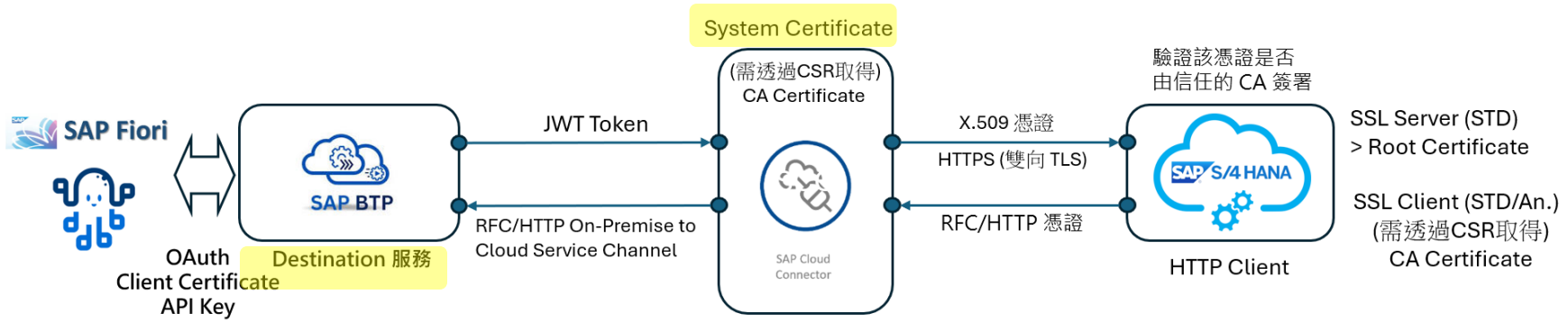
- Service Binding 運作原理：解密 VCAP_SERVICES 環境變數的注入機制與 JSON 結構。
- MTA (Multi-Target Application) 自動化綁定：如何在 mta.yaml 中定義 requires 與 resources 的依賴關係。
- Service Key 應用場景：本機開發（VS Code/BAS）、外部工具（Postman/DBeaver）的憑證核發與安全規範。
- **架構對比：Binding 與 Service Key 在安全性、生命週期與適用對象的全面對比。**

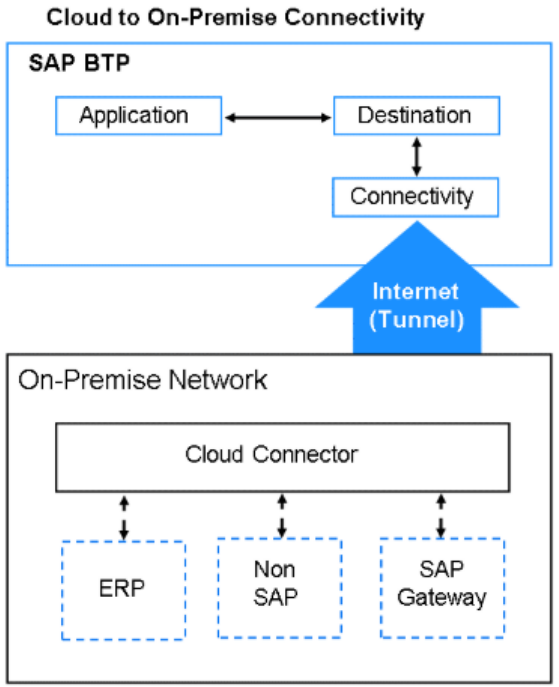
	Service Binding (服務綁定)	Service Key (服務金鑰)
主要對象	**BTP 內部的應用程式**（如 CF 上的 Pod/Container）	**BTP 外部的實體**（如本機電腦、Postman、地端系統）
生命週期	**與應用程式綁定**。程式刪除或重啟時，綁定關係隨之動態更新或消失。	**獨立存在**。除非手動在平台上刪除，否則它會永遠留在雲端。
憑證形式	**動態注入**記憶體環境變數，不留痕跡。	**靜態文字**（JSON 字串），可被複製、匯出或傳閱。
安全性	**極高**。開發與運維人員在過程中完全碰不到、也看不到密碼。	**中至低**。因人工介入（複製貼上），存在憑證外洩或被寫死在程式碼的風險。
自動化程度	**高**。可透過 mta.yaml 實現完全自動化部署與配置。	**低**。通常需要人工介入產生，或透過腳本手動提取。

4. 跨系統整合架構：Connectivity 與 Destination 服務

建立標準、可重用的連線通道

- Destination Service 機制：目標服務的動態解析、憑證託管（Basic、OAuth2、Client Certificate）。
- Connectivity Service 角色：雲端應用如何透過此服務取得代理（Proxy）設定。
- SAP BTP 憑證管理：Keystore 與 Truststore 的設定，如何安全地管理 SSL/TLS 憑證。
  - Keystore(金鑰庫)：“我是誰”(證明自己) - Private Key / Public Certificate
  - Truststore(信任庫)：“我信任誰”(驗證別人) – Root CA 根憑證 / Intermediate CA 中繼憑證





4. 跨系統整合架構：Connectivity 與 Destination 服務

建立標準、可重用的連線通道

- **Destination Service 機制：目標服務的動態解析、憑證託管（Basic、OAuth2、Client Certificate）。**
- Connectivity Service 角色：雲端應用如何透過此服務取得代理（Proxy）設定。
- SAP BTP 憑證管理：Keystore 與 Truststore 的設定，如何安全地管理 SSL/TLS 憑證。
  - Keystore(金鑰庫)：“我是誰”(證明自己) - Private Key / Public Certificate
  - Truststore(信任庫)：“我信任誰”(驗證別人) – Root CA 根憑證 / Intermediate CA 中繼憑證

專門管理連線終端（Endpoints）與安全憑證的中央倉庫，實現「動態解析」與「憑證安全託管」。

•憑證託管（三種認證方式舉例）：

- **Basic Authentication（基本憑證）**：Destination 內託管了地端 ERP 的傳統帳號密碼（如 RFC_USER / Password123）。應用程式呼叫時，BTP 會自動在 HTTP Header 加上加密後的帳密，安全送出。
- **OAuth2 Client Credentials（權杖認證）**：BTP 應用要呼叫另一個雲端 API（如 SAP S/4HANA Cloud）。Destination 託管了 Client ID 和 Client Secret。當應用發出請求，Destination Service 會自動先去向身分驗證伺服器拿 Token（權杖），再用該 Token 幫應用程式開路，全程應用程式都碰不到密碼。
- **Client Certificate（客戶端憑證認證）**：高安全性場景。Destination 託管了 BTP 專屬的數位憑證。當連線到銀行系統時，銀行系統不看帳密，只驗證這個 BTP 託管的憑證是否合法。

4. 跨系統整合架構：Connectivity 與 Destination 服務

建立標準、可重用的連線通道

- Destination Service 機制：目標服務的動態解析、憑證託管（Basic、OAuth2、Client Certificate）。
- **Connectivity Service 角色：雲端應用如何透過此服務取得代理（Proxy）設定。**
- SAP BTP 憑證管理：Keystore 與 Truststore 的設定，如何安全地管理 SSL/TLS 憑證。
  - Keystore(金鑰庫)：“我是誰”(證明自己) - Private Key / Public Certificate
  - Truststore(信任庫)：“我信任誰”(驗證別人) – Root CA 根憑證 / Intermediate CA 中繼憑證

Connectivity Service 是中介者。雲端應用程式在虛擬網路中，是無法直接通往內部私有網路（地端）的。負責分發「Proxy 鑰匙」。

舉例說明：(雲端採購應用要撈取地端 ERP 的資料)

- 採購應用會先呼叫 **Connectivity Service**，該服務會回傳一組臨時的 **Proxy 設定（包含 Proxy 主機網址與 Access Token）**。
- 採購應用將這組 Proxy 設定帶入它的網路請求中。
- 這個請求就會被導向 BTP 與地端之間的安全通道（通常是配合 SAP Cloud Connector），順利穿透企業防火牆找到地端 ERP。
- 如果沒有 Connectivity Service 提供 Proxy 設定，雲端應用的請求就會直接流向外網（Internet），進而被地端防火牆擋下。

4. 跨系統整合架構：Connectivity 與 Destination 服務

建立標準、可重用的連線通道

- Destination Service 機制：目標服務的動態解析、憑證託管（Basic、OAuth2、Client Certificate）。
- Connectivity Service 角色：雲端應用如何透過此服務取得代理（Proxy）設定。
- **SAP BTP 憑證管理：Keystore 與 Truststore 的設定，如何安全地管理 SSL/TLS 憑證。**
  - Keystore(金鑰庫)：Private Key / Public Certificate
  - Truststore(信任庫)：Root CA 根憑證 / Intermediate CA 中繼憑證

憑證管理是透過密碼學元件，確保 SSL/TLS 傳輸安全，建立雙向信任。

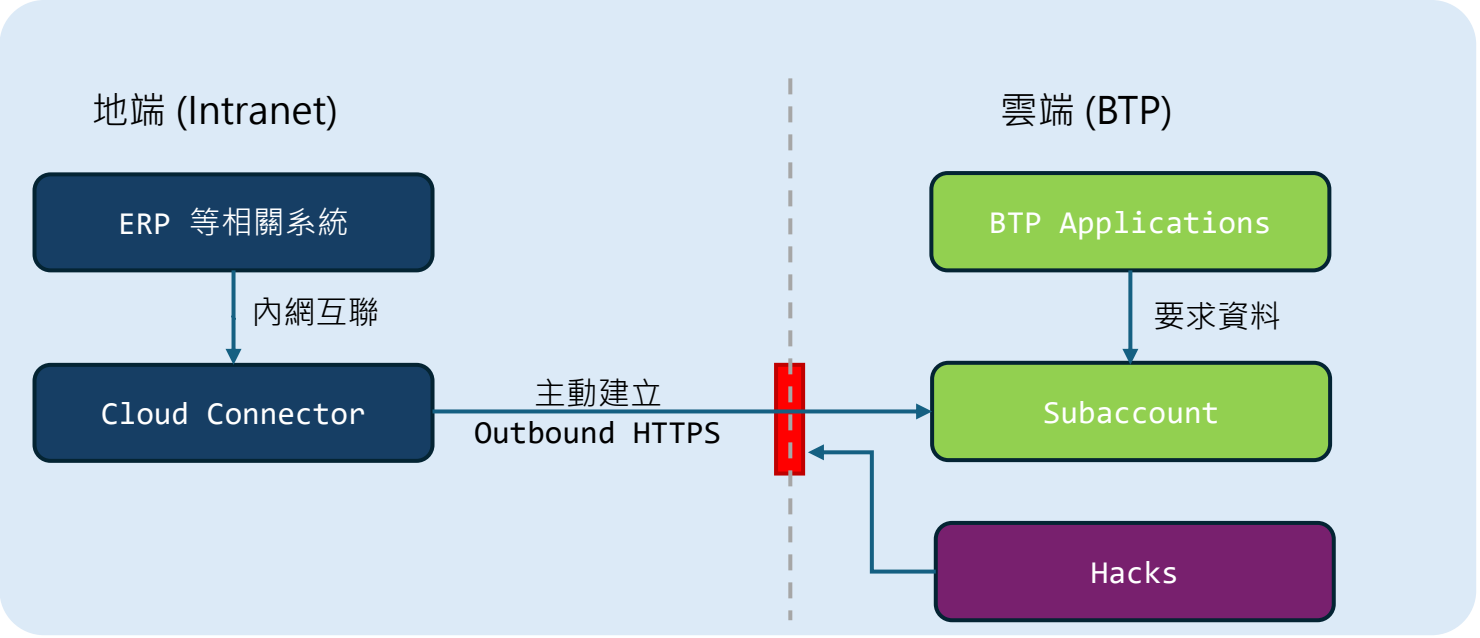
- **Keystore（金鑰庫）：“我是誰”（證明自己）包含私鑰（Private Key）與公鑰憑證（Public Certificate）。**
- **Truststore（信任庫）：“我信任誰”（驗證別人）包含根憑證（Root CA）與中繼憑證（Intermediate CA）。不含私鑰。**

```
Integration Suite -> Monitor -> Integration and APIs -> Manage Security -> Keystore
    Add -> Key Pair [就是上傳包含私鑰的憑證（屬於 Keystore 概念，證明自己）]
    Add -> Certificate [就是上傳純公鑰、根憑證或中繼憑證（屬於 Truststore 概念，信任別人）]
BTP Cockpit -> Subaccount -> Connectivity
                                    Destinations 定義外部系統的 URL、連線協定（HTTP/RFC 等）以及認證方式（Basic、OAuth2 等）
                                    Destination Certificates >> Keystore
                                    Destination Trust >> Truststore
                                    Cloud Connector >> 地端 SAP Cloud Connector（CC）與此 Subaccount 的連線狀態
```

5. 雲地整合橋樑：SAP Cloud Connector 技術深度解析

安全地打通雲端 BTP 與地端企業內網（如 SAP ERP, S/4HANA On-Premise）的連線

- **Cloud Connector (SCC) 架構：反向通道（Reverse Tunnel）技術如何做到「免開外網防火牆 inbound port」。**
- 訪問權限與安全控管：白名單（White-list）機制，精細控制可存取的 On-Premise 主機、Port 及 URL 路徑。
- 主體傳播 (Principal Propagation)：如何將雲端使用者的身份（Azure AD/IdP），安全地傳遞至地端 SAP 系統實現單一登入 (SSO)。



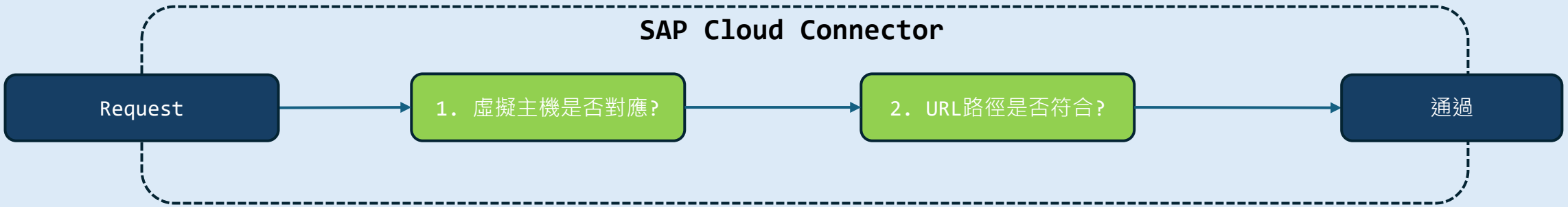
5. 雲地整合橋樑：SAP Cloud Connector 技術深度解析

安全地打通雲端 BTP 與地端企業內網（如 SAP ERP, S/4HANA On-Premise）的連線

- Cloud Connector (SCC) 架構：反向通道（Reverse Tunnel）技術如何做到「免開外網防火牆 inbound port」。
- **訪問權限與安全控管：白名單（White-list）機制，精細控制可存取的 On-Premise 主機、Port 及 URL 路徑。**
- 主體傳播 (Principal Propagation)：如何將雲端使用者的身份（Azure AD/IdP），安全地傳遞至地端 SAP 系統實現單一登入 (SSO)。

SCC 扮演了極其嚴格的「門衛」角色，採用**預設全部阻斷（Deny All）**，只允許**白名單（Allow List）**的極簡原則。

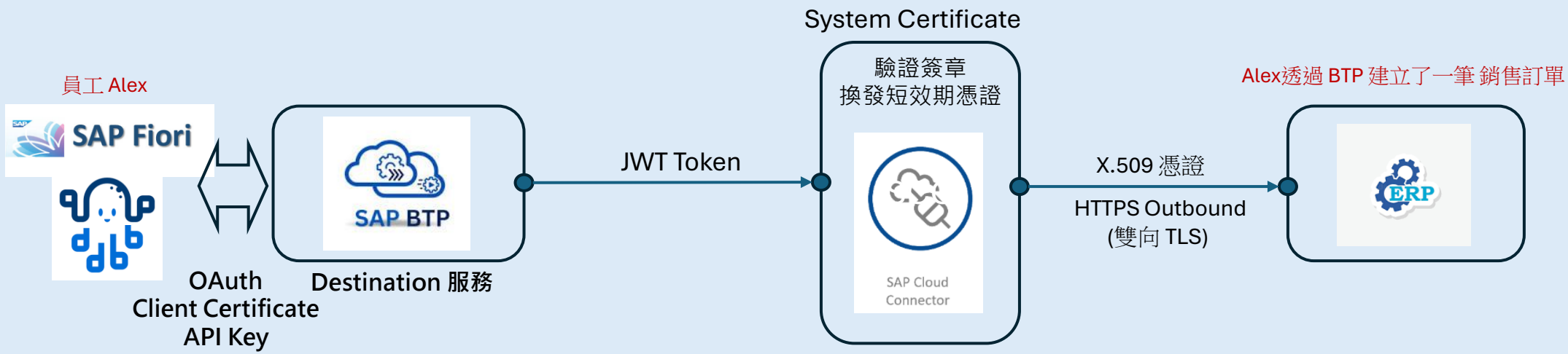
IT 網管可以在 SCC 中進行三層精細控制：**主機（Host） -> 通訊埠（Port）-> URL 路徑**。



5. 雲地整合橋樑：SAP Cloud Connector 技術深度解析

安全地打通雲端 BTP 與地端企業內網（如 SAP ERP, S/4HANA On-Premise）的連線

- Cloud Connector (SCC) 架構：反向通道（Reverse Tunnel）技術如何做到「免開外網防火牆 inbound port」。
- 訪問權限與安全控管：白名單（White-list）機制，精細控制可存取的 On-Premise 主機、Port 及 URL 路徑。
- **主體傳播 (Principal Propagation)：如何將雲端使用者的身份（Azure AD/IdP），安全地傳遞至地端 SAP 系統實現單一登入 (SSO)。**



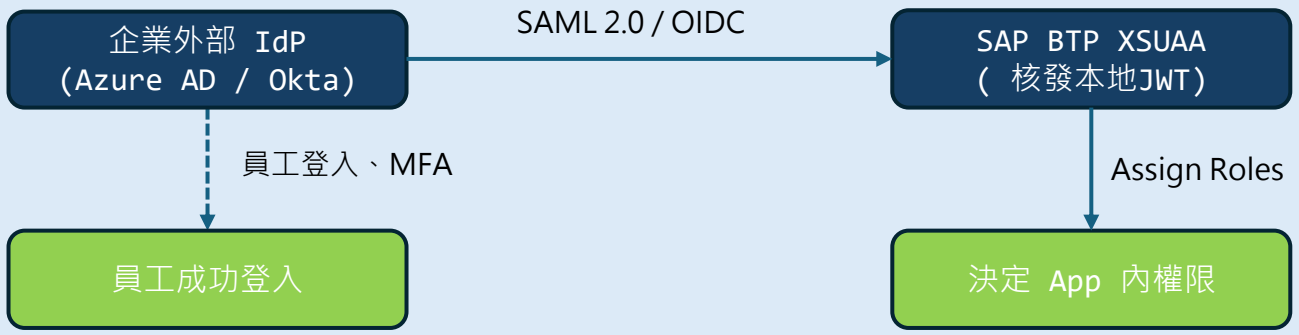
安全審計：打開地端 ERP 的 SUIM 或 ST03N 系統日誌（Log），會清清楚楚記錄著：「2026/07/20員工 Alex透過 BTP 建立了一筆 100 萬元的銷售訂單」，符合企業內控與資安合規性。

6. Service Key

企業的身分驗證系統並進行細粒度的權限控管

- **XSUAA 服務深度剖析：SAP 核心身分驗證服務的架構，與外部身分提供者 (IdP, 如 Azure AD、Okta) 的 SAML 2.0 / OIDC 整合。**
- OAuth 2.0 授權流程：理解 JWT (JSON Web Token) 在 BTP 應用微服務之間的傳遞與驗證。
- 應用程式安全定義 (xs-security.json)：如何定義 Scopes、Roles 與 Role Collections，並在程式碼中執行權限檢查。

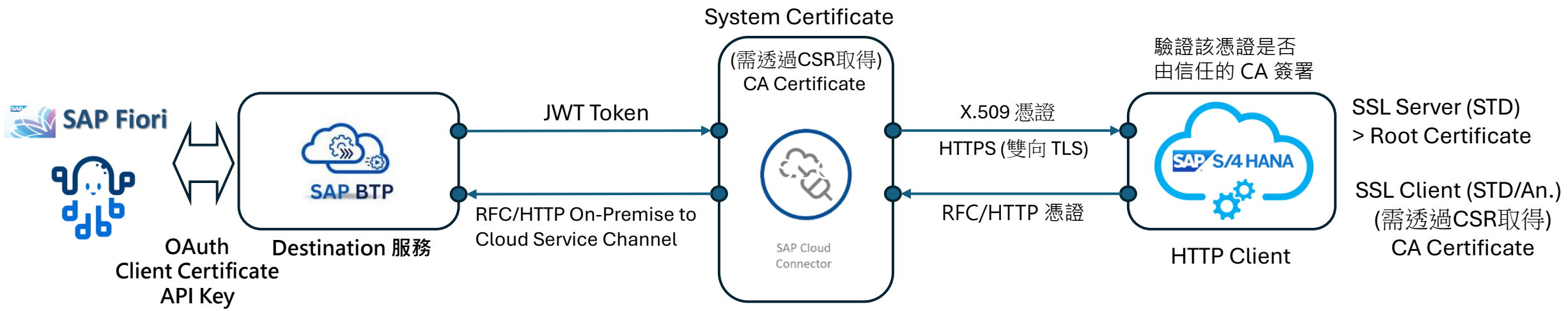
XSUAA 是 SAP 基於開源 Cloud Foundry UAA 所延伸的專屬身份驗證與授權服務。



6. Service Key

企業的身分驗證系統並進行細粒度的權限控管

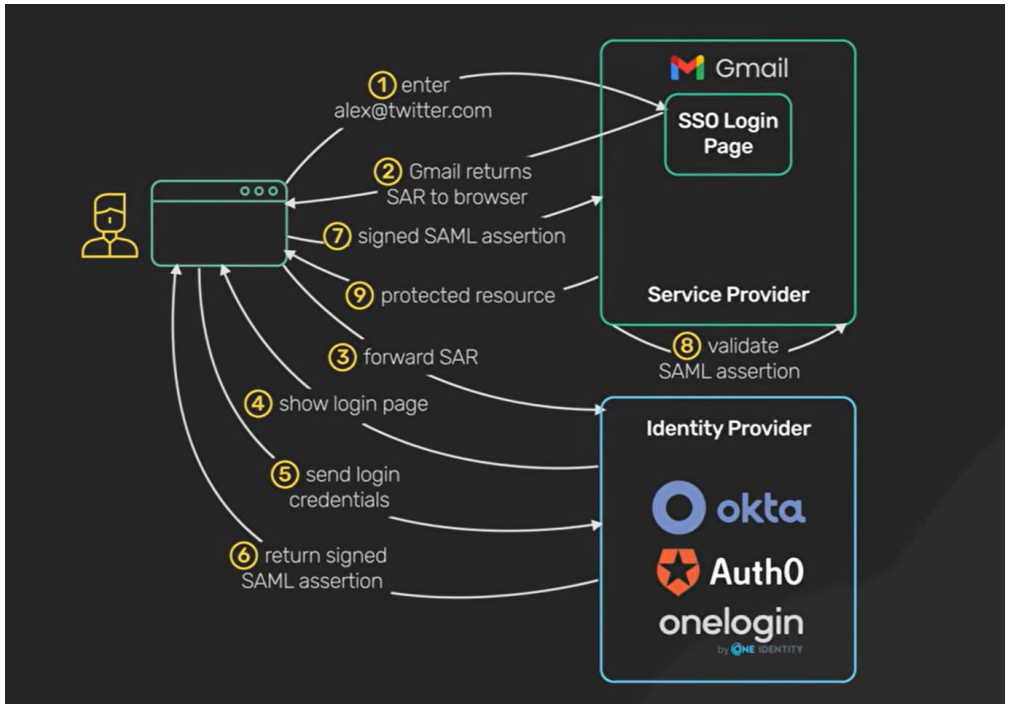
- XSUAA 服務深度剖析：SAP 核心身分驗證服務的架構，與外部身分提供者 (IdP, 如 Azure AD、Okta) 的 SAML 2.0 / OIDC 整合。
- **OAuth 2.0 授權流程：理解 JWT (JSON Web Token) 在 BTP 應用微服務之間的傳遞與驗證。**
- 應用程式安全定義 (xs-security.json)：如何定義 Scopes、Roles 與 Role Collections，並在程式碼中執行權限檢查。



6. Service Key

企業的身分驗證系統並進行細粒度的權限控管

- XSUAA 服務深度剖析：SAP 核心身分驗證服務的架構，與外部身分提供者 (IdP, 如 Azure AD、Okta) 的 SAML 2.0 / OIDC 整合。
- **OAuth 2.0 授權流程：理解 JWT (JSON Web Token) 在 BTP 應用微服務之間的傳遞與驗證。**
- 應用程式安全定義 (xs-security.json)：如何定義 Scopes、Roles 與 Role Collections，並在程式碼中執行權限檢查。



6. Service Key - OAuth 2.0 授權流程：理解 JWT (JSON Web Token) 在 BTP 應用微服務之間的傳遞與驗證。

1. 輸入電子郵件

使用者在服務提供者（SP，此處為 Gmail）輸入電子郵件（如 `alex@twitter.com`）。系統會根據網域識別出該帳號隸屬於某企業，必須透過該企業的企業認證系統進行登入。

2. 回傳 SAR 至瀏覽器

Gmail（服務提供者）生成一個 **SAML 驗證請求 (SAML AuthnRequest，圖中簡稱 SAR)**，其中包含目標 IdP 地址和安全參數，並將其回傳給使用者的瀏覽器。

3. 轉發 SAR

瀏覽器收到 SAR 後，會透過網頁重定向（Redirect）自動將這個 **SAML 請求轉發**給指定的身份驗證提供者（IdP，如 Okta / Auth0 / OneLogin）。

4. 顯示登入頁面

IdP 接收到請求並確認無誤後，向使用者的瀏覽器**顯示企業專屬的登入介面**，準備收集使用者的身份憑證。

5. 發送登入憑證

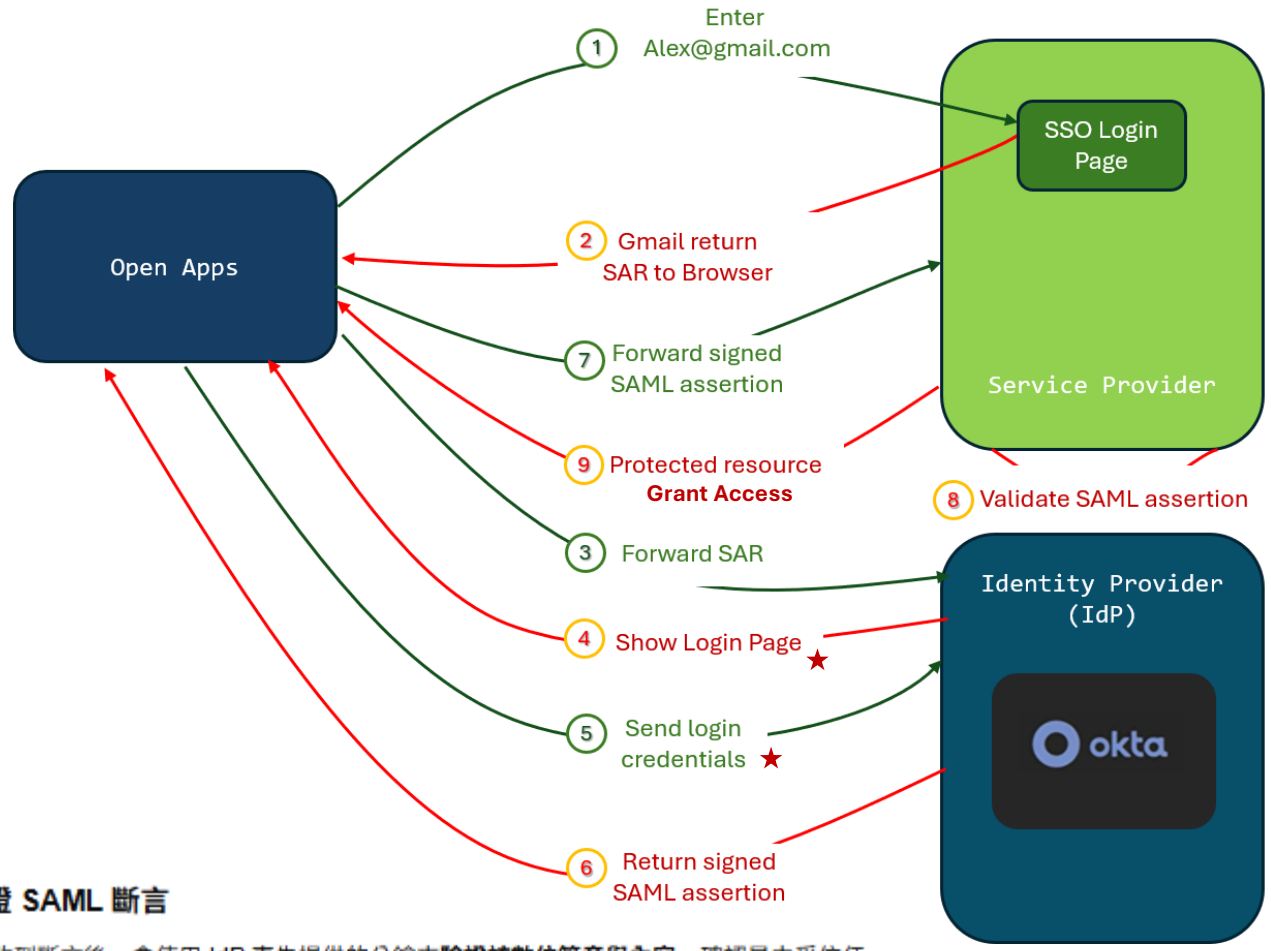
使用者在介面上輸入使用者名稱、密碼（或進行多因素驗證 MFA），瀏覽器將這些**登入憑證安全地發送**給 IdP 進行檢驗。

6. 回傳已簽署的 SAML 斷言

IdP 成功驗證使用者憑證後，會生成一個包含使用者身份資訊的 **SAML 斷言 (SAML Assertion)**。IdP 會使用自己的私鑰對其進行**數位簽署**（確保不被竄改），並將其回傳給瀏覽器。

7. 發送已簽署的 SAML 斷言

瀏覽器再度透過自動表單提交（POST）或重定向，將這個帶有數位簽章的 **SAML 斷言發送**回 Gmail（服務提供者）。



8. 驗證 SAML 斷言

Gmail 收到斷言後，會使用 IdP 事先提供的公鑰來**驗證該數位簽章與內容**。確認是由受信任的 IdP 所發出且內容無誤後，即代表信任該使用者身份。

9. 建立會話並正式登入 (最後隱藏步驟)

Gmail 驗證通過後，在伺服器端為該使用者**建立登入會話 (Session)** 並發放 Cookie。接著，瀏覽器自動**重新導向至使用者的 Gmail 收件匣**，使用者便能正式開始使用服務。

6. Service Key

企業的身分驗證系統並進行細粒度的權限控管

- XSUAA 服務深度剖析：SAP 核心身分驗證服務的架構，與外部身分提供者 (IdP, 如 Azure AD、Okta) 的 SAML 2.0 / OIDC 整合。
- OAuth 2.0 授權流程：理解 JWT (JSON Web Token) 在 BTP 應用微服務之間的傳遞與驗證。
- **應用程式安全定義 (xs-security.json)：如何定義 Scopes、Roles 與 Role Collections，並在程式碼中執行權限檢查。**

•**核心三要素定義**：**Scope（範圍）**：程式碼層級的最小權限單元（如：能讀、能寫）。

•**Role Template（角色模板）**：將多個 Scopes 打包成一個模板（如：經理模板包含讀與寫）。

•**Role Collection（角色群組）**：在 BTP Cockpit 網頁上呈現的最終包裝，可以指派給特定員工（如：Finance_Manager_Collection）。

json

```
{
  "xsappname": "myapp",
  "tenant-mode": "dedicated",
  "scopes": [
    {
      "name": "$XSAPPNAME.PurchaseOrder.Read",
      "description": "允許讀取採購單"
    },
    {
      "name": "$XSAPPNAME.PurchaseOrder.Write",
      "description": "允許建立與修改採購單"
    }
  ],
  "role-templates": [
    {
      "name": "ViewerRole",
      "description": "一般檢視者",
      "scope-references": ["$XSAPPNAME.PurchaseOrder.Read"]
    },
    {
      "name": "ManagerRole",
      "description": "高階主管",
      "scope-references": ["$XSAPPNAME.PurchaseOrder.Read", "$XSAPPNAME.Purchase(
    }
  ]
}
```

javascript

```
// 使用 SAP CAP (Cloud Application Programming) 框架的權限檢查範例
service.on('UPDATE', 'PurchaseOrders', async (req) => {

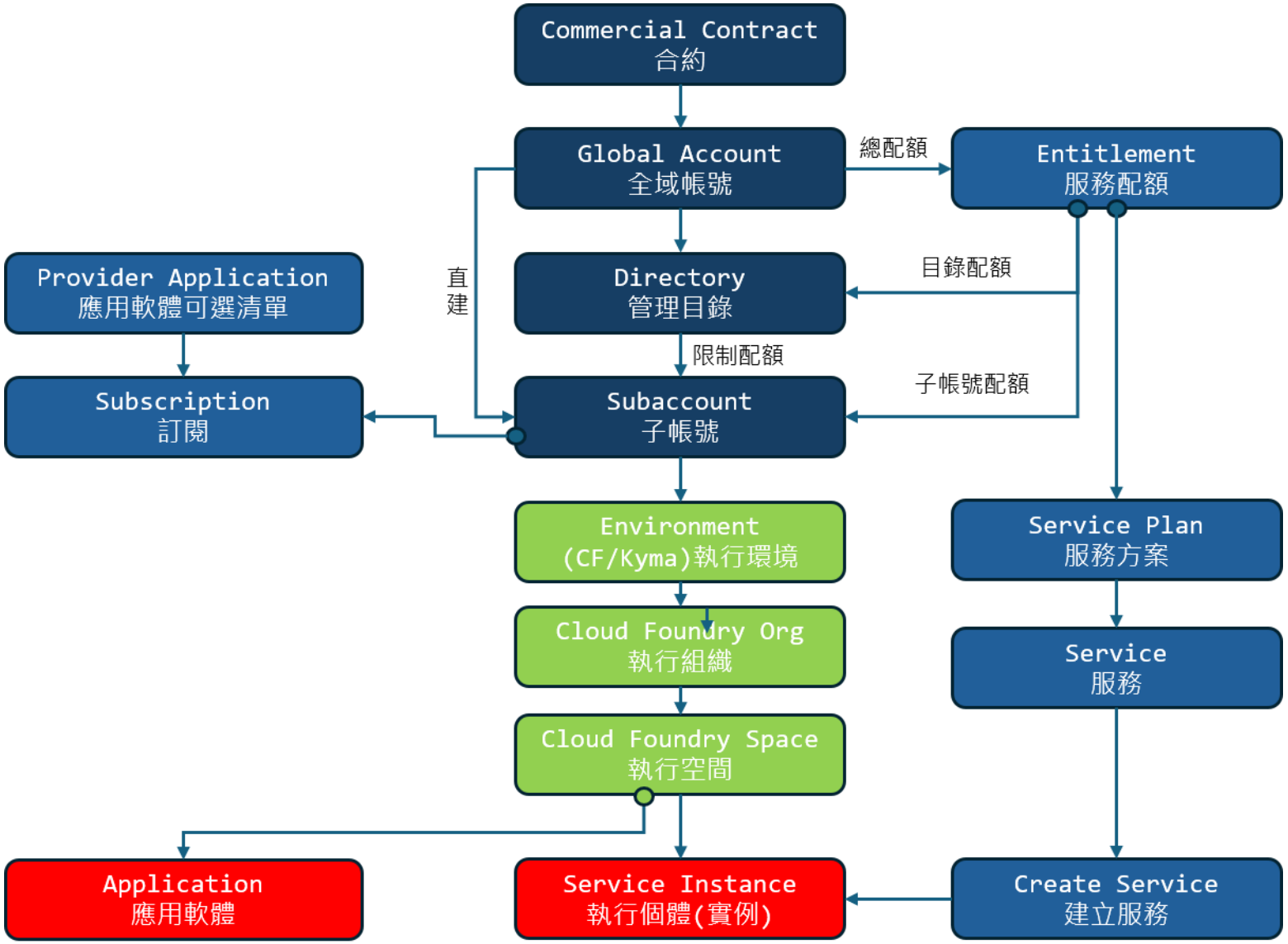
    // 🔍 1. 檢查用戶是否擁有特定 Scope (xs-security.json 中定義的 Write 權限)
    if (!req.user.hasScope('PurchaseOrder.Write')) {
        return req.error(403, '🔒 拒絕存取：您沒有修改採購單的權限！');
    }

    // 🧱 2. 進一步執行數據層級的細粒度控管 (Data-level security)
    const userCompanyCode = req.user.attr.co_code; // 從 JWT 的屬性中撈取公司別
    const targetOrder = req.data.ID;

    // 檢查該採購單的公司別是否與用戶相符...
    if (userCompanyCode !== '1000') {
        return req.error(403, '🔒 拒絕存取：您無權操作此公司別的數據！');
    }

    // 通過所有檢查，執行資料庫更新...
});
```

Q&A


Commercial Contract
合約
Global Account
全域帳號
總配額
Entitlement
服務配額
Provider Application
應用軟體可選清單
直建
Directory
管理目錄
目錄配額
限制配額
子帳號配額
Subscription
訂閱
Subaccount
子帳號
Environment
(CF/Kyma)執行環境
Service Plan
服務方案
Cloud Foundry Org
執行組織
Service
服務
Cloud Foundry Space
執行空間
Application
應用軟體
Service Instance
執行個體(實例)
Create Service
建立服務